

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogLeNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogLeNet (Inception Network)

GoogLeNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogLeNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

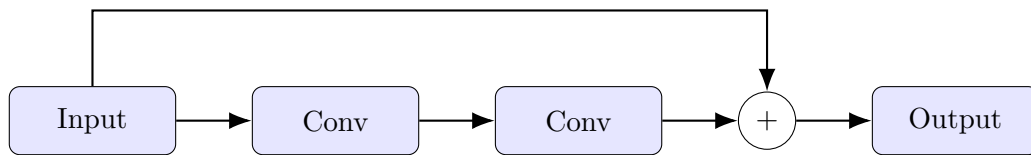


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis

- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

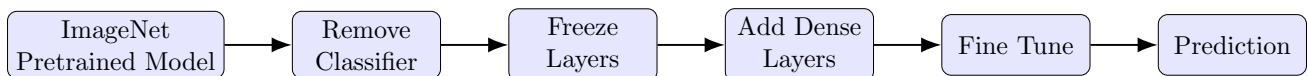


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Source Code: The complete notebook and code for this experiment is available at https://github.com/Lunarya-git/Deep-Learning-Experiments/tree/main/Experiment_4.

Task 1: Dataset Preparation

The CIFAR-10 dataset was loaded directly using `tensorflow.keras.datasets.cifar10`. The pixel values (originally 0–255) were converted to `float32` and divided by 255 so that every pixel lies between 0 and 1. This is a standard step because neural networks train faster and more reliably when the input values are small and in a consistent range.

The original 50,000 training images were further split into a 90:10 train/validation split (45,000 training images and 5,000 validation images), keeping the original 10,000 images as the test set. The split was done using `train_test_split` with stratification, so all 10 classes remain balanced in each subset. Labels were one-hot encoded using `to_categorical` since the model uses a softmax output with categorical cross-entropy loss.

- Training set shape: $45000 \times 32 \times 32 \times 3$
- Validation set shape: $5000 \times 32 \times 32 \times 3$
- Testing set shape: $10000 \times 32 \times 32 \times 3$
- Number of classes: 10

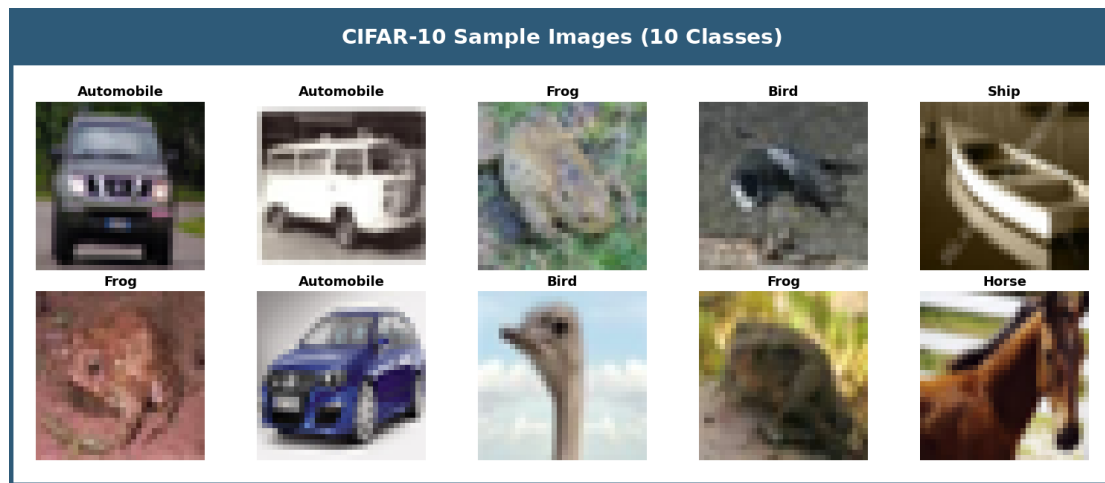


Figure 3: Ten sample images from the CIFAR-10 training set

Inference: The sample images confirm that the dataset was loaded and labelled correctly – each small 32×32 image clearly corresponds to its printed class name. Because the images are so small, some classes (like cat vs. dog, or automobile vs. truck) look visually similar even to a human eye, which hints that the model may confuse these pairs later on.

Task 2: Transfer Learning

For this experiment, **VGG16** was chosen as the pretrained backbone. VGG16 was loaded with `weights='imagenet'` and `include_top=False`, which removes the original 1000-class ImageNet classifier and keeps only the convolutional feature-extraction layers. Since the CIFAR-10 images are already $32 \times 32 \times 3$, no resizing was required for VGG16 (it accepts this size directly, unlike GoogleNet/InceptionV3, more on this in the Results section).

Before feeding images to VGG16, they were passed through the `preprocess_input` function from Keras' VGG16 applications module, which rescales/reorders the channels in the same way VGG16 was originally trained on ImageNet.

The convolutional base was frozen (`base_model.trainable = False`) so that only the new layers learn during the first phase of training. On top of the frozen base, the following classifier head was added:

1. Global Average Pooling layer – converts each feature map into a single number, drastically reducing the number of parameters compared to flattening.
2. Dense layer (128 units, ReLU activation).
3. Output Dense layer (10 units, Softmax activation) for the 10 CIFAR-10 classes.

The model was compiled with the Adam optimizer and categorical cross-entropy loss, tracking accuracy as the evaluation metric.

Task 3: Model Training

The model was trained using the following settings.

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	15
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Training took about **3.18 minutes** on GPU for 15 epochs. Training accuracy kept climbing steadily (from 52.9% to 83.75%), but validation accuracy plateaued around 64–65% quite early (by epoch 6) and even dipped slightly afterwards. This is a classic sign that the frozen-base model was starting to overfit the training data – the top layers were memorising patterns rather than learning anything new, since the convolutional base itself was not being updated. The full accuracy/loss curves are shown together with the fine-tuning phase in Section 17 (Mandatory Plots).

Task 4: Fine Tuning

For fine-tuning, only **Block 5** of VGG16 (the last convolutional block: `block5_conv1`, `block5_conv2`, `block5_conv3` and `block5_pool`) was unfrozen, while all earlier blocks stayed frozen. This gave 7,146,378 trainable parameters and 7,635,264 non-trainable parameters. The model was recompiled with a much smaller learning rate (1×10^{-5}) so that the pretrained ImageNet weights are only nudged gently and not destroyed, and trained for 8 more epochs (2.49 minutes).

Best Validation Accuracy Before Fine-Tuning	64.96%
Best Validation Accuracy After Fine-Tuning	69.82%
Improvement	+4.86%

Fine-tuning clearly helped – validation accuracy went up by almost 5 percentage points. However, training accuracy shot up to 97.4% while validation loss kept climbing throughout fine-tuning (from 1.58 to 1.76), so the model is also overfitting more once its own weights start moving. This trade-off (higher accuracy but more overfitting) is discussed further in Section 19.

Task 5: Model Evaluation

After fine-tuning, the model was evaluated once on the untouched 10,000-image CIFAR-10 test set.

Testing Accuracy	69.76%
Precision (Macro)	69.75%
Recall (Macro)	69.76%
F1-score (Macro)	69.70%

Looking at the per-class classification report, "Ship" (80.6% F1) and "Automobile" (79.3% F1) were the easiest classes to recognise, while "Cat" (52.8% F1) was by far the hardest – the model often confused cats with dogs, which makes sense given how visually similar small 32×32 images of these two animals can be. The confusion matrix (Section 17) shows this clearly.

16. Hyperparameter Study

Eight configurations were trained (frozen VGG16 base, 10 epochs unless noted) to see how each hyperparameter affects validation accuracy. All results are summarised below and sorted from best to worst.

Optimizer	LR	Batch	Dense	Epochs	Frozen Layers	Best Val. Acc. (%)
Adam	0.001	64	128	10	All	65.52
Adam	0.001	32	128	10	All	65.28
Adam	0.001	32	256	10	All	65.10
Adam	0.001	32	128	20	All	64.70
Adam	0.001	16	128	10	All	64.68
SGD	0.001	32	128	10	All	62.62
Adam	0.0001	32	128	10	All	61.66
Adam	0.001	32	128	10	Partial	10.00

Hyperparameter Study and Architecture-wise Comparison

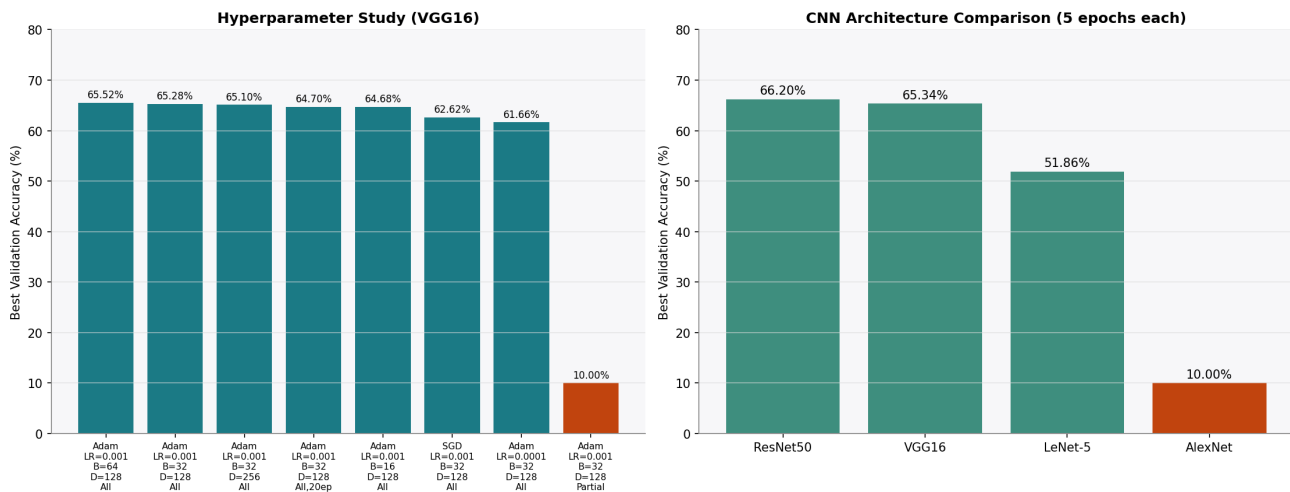


Figure 4: Left: hyperparameter study results for VGG16 (8 configurations). Right: comparison of the four CNN architectures on CIFAR-10.

Inference: The differences between most configurations are small (within 1–4%), so learning rate, batch size, and dense units only made minor differences here. The one big outlier is the last row, where only Block 5 was partially unfrozen *before* the classifier head had learned anything useful – accuracy collapsed to 10%, which is exactly what random guessing would give for 10 classes. This tells us fine-tuning too early (before the new classifier layers are trained) can break the model rather than help it, which is why Task 4 always trains the frozen model first and unfreezes later.

17. Mandatory Plots

17.1 Sample CIFAR-10 Images

See Figure 3 in Section 15 (Task 1). The images look correctly labelled, and their small size explains why some classes get confused later.

17.2 Training and Validation Accuracy, Training and Validation Loss

Since these four plots all tell one continuous story (main training followed immediately by fine-tuning), they are shown together as one 2×2 figure spanning all 23 epochs (epochs 1–15 are the frozen-base phase, epochs 16–23 are the fine-tuning phase). A dashed grey line marks where fine-tuning begins.

VGG16 Transfer Learning: Accuracy and Loss (Epochs 1-15 main training, 16-23 fine-tuning)

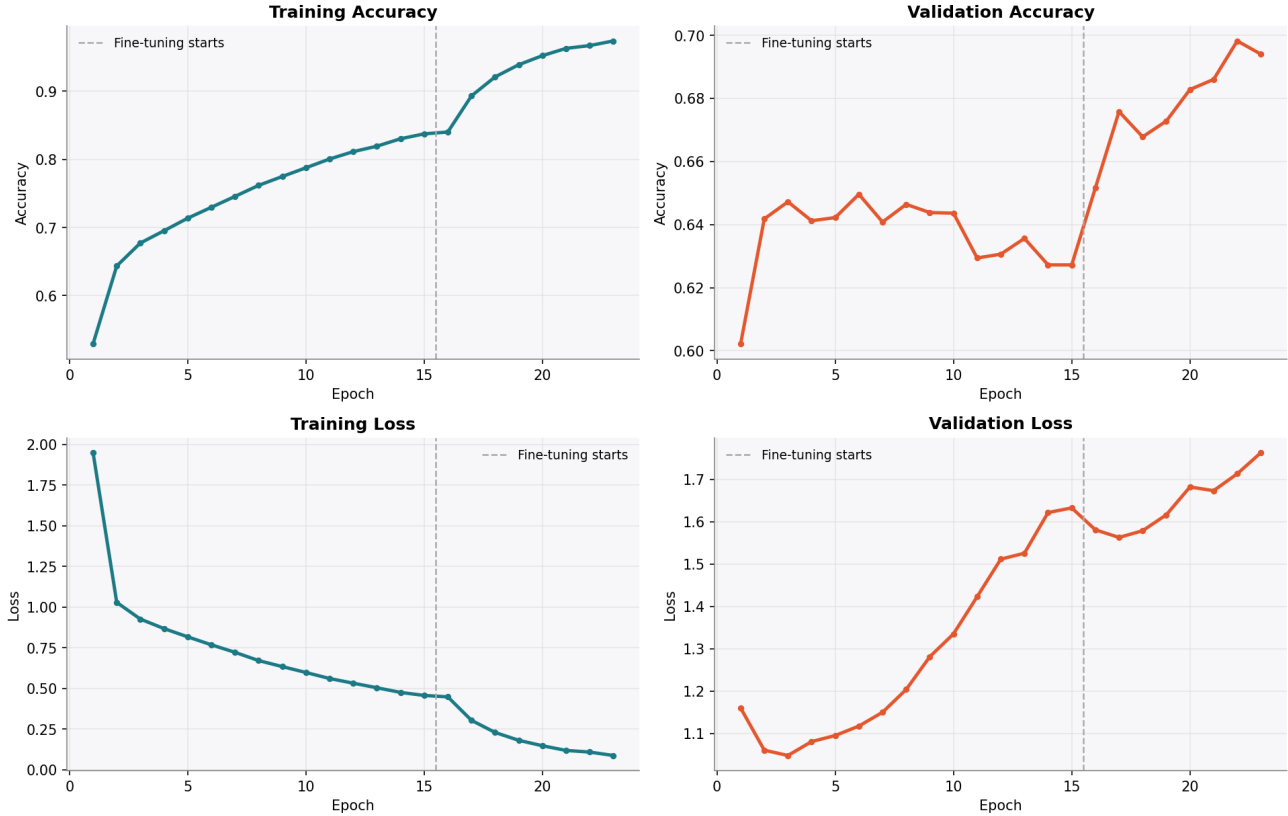


Figure 5: Training/validation accuracy and loss across the full 23-epoch run (teal = training, coral = validation). The dashed line marks the start of fine-tuning.

Inference: Training accuracy rises smoothly throughout, reaching 97.4% by the end, while training loss keeps falling – the model is clearly able to fit the training data very well. Validation accuracy behaves very differently: it plateaus around 63–65% during the frozen phase, then jumps up once fine-tuning starts, reaching a best of 69.82%. But validation loss keeps *increasing* for most of the run, even while validation accuracy improves during fine-tuning. This gap between training and validation performance is a textbook overfitting pattern – the model memorises training examples faster than it learns features that generalise to unseen images.

17.3 Fine-Tuning Impact and Optimizer Comparison

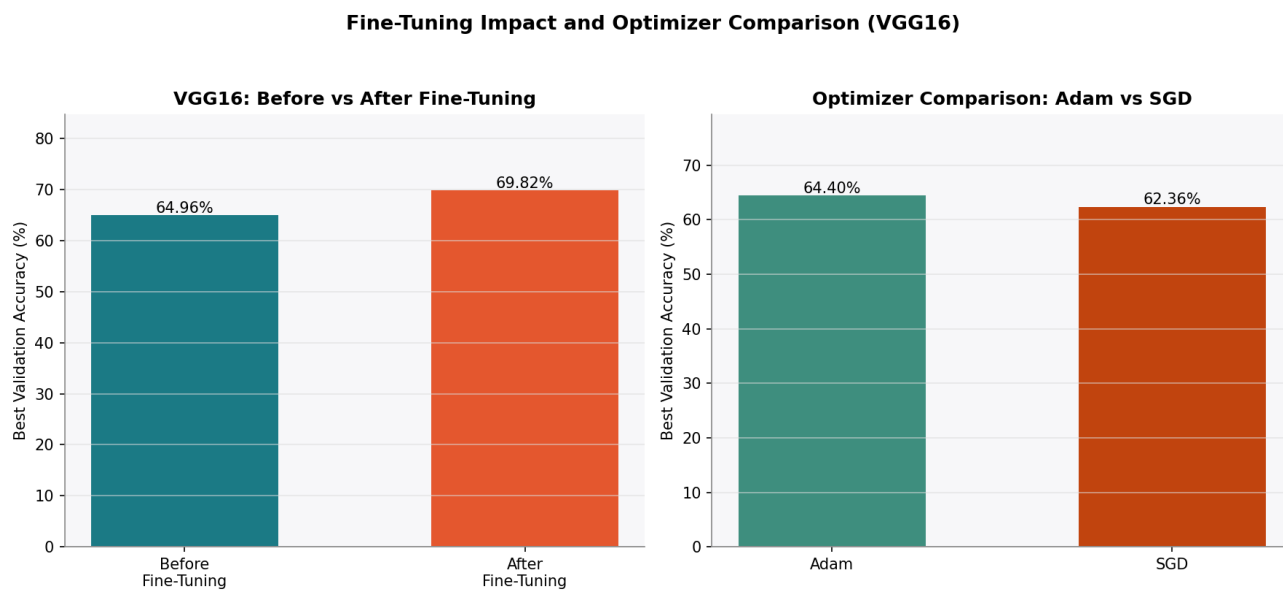


Figure 6: Left: validation accuracy before vs. after fine-tuning. Right: Adam vs. SGD optimizer comparison, both trained for 10 epochs with the frozen VGG16 base.

Inference: Fine-tuning the last convolution block clearly helps, adding about 5% accuracy. Between optimizers, Adam (64.40%) beat SGD (62.36%) under identical settings – Adam adapts its learning rate per parameter, so it converges faster within a fixed number of epochs, while plain SGD needs more epochs (or momentum/scheduling) to catch up.

17.4 Confusion Matrix

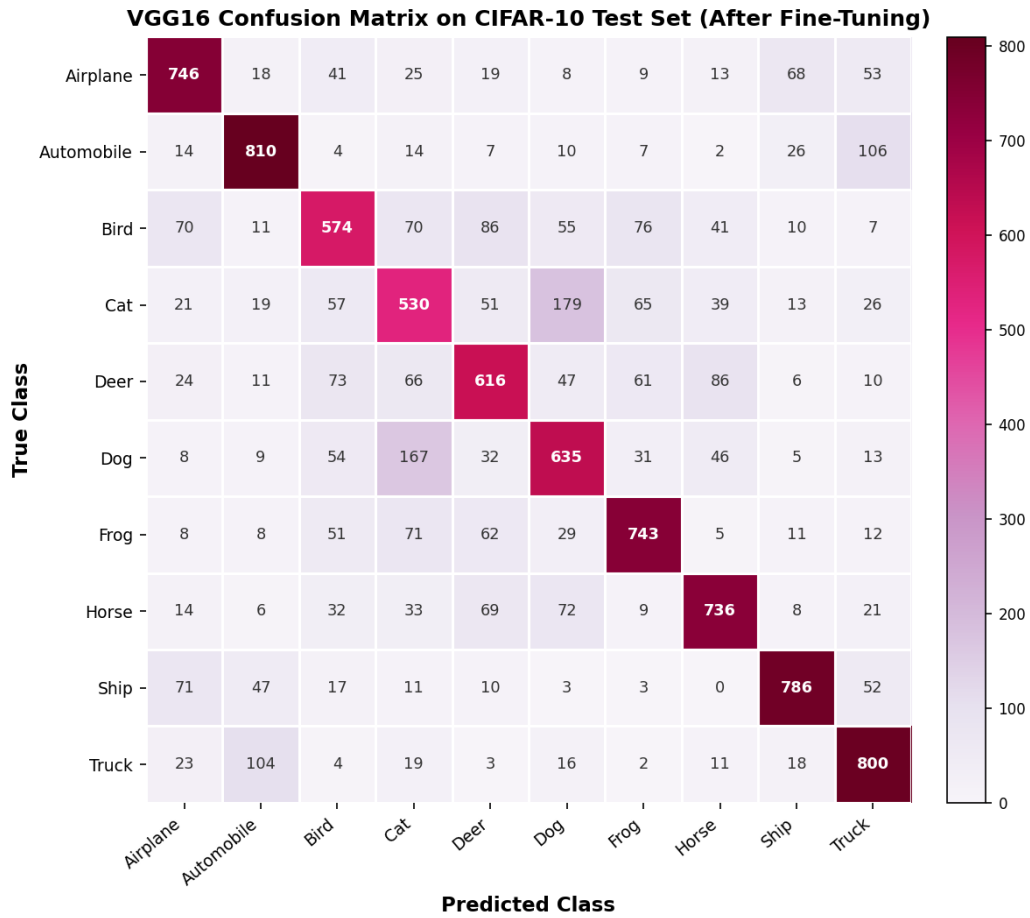


Figure 7: Confusion matrix of the fine-tuned VGG16 model on the CIFAR-10 test set

Inference: The diagonal is strongly dominant for most classes, meaning the model gets most predictions right. The biggest confusion is between *Cat* and *Dog* (179 cats predicted as dogs, 167 dogs predicted as cats), and between *Automobile* and *Truck* (106 and 104 mix-ups respectively). Both pairs are visually and semantically close categories, so this confusion makes intuitive sense even for a human looking at tiny 32×32 pictures.

17.5 Misclassified Images (Optional)

This optional plot was not generated in this run; the confusion matrix above already captures which class-pairs the model struggles with most (cat/dog and automobile/truck).

18. Results

18.1 Performance Metrics (VGG16, after fine-tuning)

Metric	Value
Best Training Accuracy	97.39%
Best Validation Accuracy (Before Fine-Tuning)	64.96%
Best Validation Accuracy (After Fine-Tuning)	69.82%
Testing Accuracy	69.76%
Precision (Macro)	69.75%
Recall (Macro)	69.76%
F1-score (Macro)	69.70%
Total Parameters	14,781,642
Initial Training Time	3.18 minutes
Fine-Tuning Time	2.49 minutes

18.2 Comparison of CNN Architectures

To compare architectures fairly, LeNet-5, AlexNet, VGG16 (transfer learning) and ResNet50 (transfer learning) were each trained for 5 epochs on the same CIFAR-10 data. GoogleNet (InceptionV3) could not be included – see the note below the table.

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	83,126	51.86	0.54 min
AlexNet	36,925,322	10.00	2.72 min
VGG16	14,781,642	65.34	1.12 min
GoogleNet	—	Not run	—
ResNet50	23,851,274	66.20	1.54 min

Note on GoogleNet: GoogleNet/InceptionV3 requires a minimum input size of 75×75 pixels, but CIFAR-10 images are only 32×32 pixels. Loading `InceptionV3` directly on CIFAR-10 raises a `ValueError` (*"Input size must be at least 75x75"*), so GoogleNet was left out of the direct architecture comparison rather than resizing every image (which would have made the comparison unfair and much slower to train).

Note on AlexNet: In this run, the AlexNet-style model got stuck at 10% accuracy (equal to random guessing among 10 classes) and the loss stayed flat near $\ln(10) \approx 2.303$ for all 5 epochs. This is a known failure mode for AlexNet-style networks with a fairly high initial learning rate and no normalization layers – the network essentially never starts learning (a "dead" training run). With a smaller learning rate, batch normalization, or a couple more careful initialisation tweaks, AlexNet should be able to learn on CIFAR-10 – but as trained here, it did not.

ResNet50 achieved the best short-training accuracy (66.20% in just 5 epochs), closely followed by VGG16 (65.34%), confirming that residual connections and pretrained ImageNet features give a strong head start compared to training a small network like LeNet-5 from scratch (51.86%).

19. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet was the first deep CNN to win the ImageNet competition by a huge margin (2012), and it proved that deep networks can actually be trained well in practice. It popularised ReLU activation (much faster than sigmoid/tanh), Dropout to reduce overfitting, and training on GPUs, which made deeper networks practically possible for the first time. Interestingly, in our own AlexNet run on CIFAR-10 the model failed to learn at all (stuck at 10% accuracy) with the settings used – a good reminder that even a "proven" architecture is sensitive to learning rate and initialization.

2. Why does VGG16 use only 3×3 convolution filters?

Two stacked 3×3 filters cover the same receptive field as one 5×5 filter, but use fewer parameters and add an extra non-linearity (ReLU) in between, making the network more expressive while staying computationally cheaper. Using one filter size everywhere also keeps the architecture simple and easy to build/scale.

3. Explain the advantages of the Inception module.

The Inception module runs 1×1 , 3×3 , 5×5 convolutions and pooling all in parallel on the same input and then concatenates the results, so the network can capture features at multiple scales at once. The 1×1 convolutions before the bigger filters reduce the number of channels first, which keeps the parameter count and computation much lower than a plain VGG-style network of similar depth.

4. What is the purpose of residual learning?

Residual (skip) connections let the network learn a small correction $F(x)$ on top of the input x , instead of learning the entire mapping $H(x)$ from scratch. This makes it much easier for gradients to flow backward during training, which solves the vanishing-gradient problem and allows networks to be made very deep (50, 101, even 152 layers) without training becoming unstable.

5. Differentiate LeNet and ResNet.

LeNet-5 is a tiny 7-layer network from 1998 with only about 60K–83K parameters, using tanh activations and average pooling – good enough for simple digit recognition but far too shallow for complex natural images. ResNet is a much deeper (50+ layer), modern architecture using ReLU and residual connections, with millions of parameters, capable of learning far richer features. In our own comparison, LeNet-5 reached only 51.86% on CIFAR-10 in 5 epochs while ResNet50 reached 66.20% in the same 5 epochs.

6. What is Transfer Learning?

Transfer Learning means taking a CNN that has already been trained on a large dataset (like ImageNet) and reusing its learned features for a new, usually smaller, dataset – instead of training a network from scratch. Typically the pretrained convolutional base is frozen and only a new classifier head is trained on top of it, as was done with VGG16 in this experiment.

7. Why is fine tuning required?

A frozen pretrained model only adapts its new classifier layers – the underlying features are still tuned for ImageNet, not for the new dataset. Fine-tuning unfreezes some of the later convolutional layers and trains them (with a small learning rate) so the features shift slightly to fit the new data better. In our experiment, fine-tuning Block 5 of VGG16 improved validation accuracy from 64.96% to 69.82%.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated (atrous) convolution inserts gaps between kernel elements to enlarge the receptive field *without* increasing the number of parameters or changing the output size – useful for tasks like semantic segmentation where context matters. Transpose convolution does the opposite job: it *increases* the spatial size of a feature map (learnable upsampling), which is needed in tasks like image generation, super-resolution, or the decoder half of an autoencoder/GAN.

9. Why do pretrained models converge faster?

Pretrained models already know general-purpose visual features (edges, textures, shapes, simple object parts) learned from a huge dataset like ImageNet. The new task only needs to adjust or add a small classifier on top of these features rather than learning everything from random weights, so far fewer updates are needed to reach a good accuracy – which is exactly why the VGG16

transfer-learning model in this experiment reached 60%+ accuracy within the very first epoch, while LeNet-5 and AlexNet (trained from scratch) needed many more epochs to get anywhere close.

10. Compare the computational complexity of LeNet and ResNet.

LeNet-5 is extremely lightweight (about 60K parameters, 7 layers), so it trains and runs very fast – in our run it finished 5 epochs in just 0.54 minutes. ResNet50 has around 23.8 million parameters and 50 layers, so it needs far more computation (matrix multiplications, memory) per image – it took 1.54 minutes for the same 5 epochs on the same hardware. ResNet50's extra complexity clearly pays off in accuracy (66.20% vs. 51.86%), but at a real cost in training time, model size and memory usage.

20. Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.
4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

21. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>
9. Source Code for this Experiment: https://github.com/Lunarya-git/Deep-Learning-Experiments/tree/main/Experiment_4